TAKE-HOME ASSIGNMENT

Access Control Service

Design and build the core of a service that decides who may do what on which job.

The Situation

NFPL is a customs brokerage with about **120 staff across 5 branches**. Each shipment is a **job**. A job belongs to one client, one branch, one direction (import or export) and one mode (air, FCL or LCL).

A job moves through stages, roughly **Confirmed → Prepared → Checked → Filed**. It can also be **Cancelled** or **Reopened**. Different people do the work at each stage. Some staff handle many clients; some handle only a narrow slice of the business.

Last month a filed shipment was altered by someone who had no business touching it. Nobody could say who had been allowed to, or why. Management wants an access system that is **strict where it matters and still easy to run day to day**.

Your Task

Build a service that decides who may do what on which job. At a minimum it should:

- Let someone with the right authority **manage who has what**.
- Answer whether a given person **may perform a given action on a given job**, and explain the answer.
- Show a person **the jobs that need them**.
- Keep a **history** that supports the questions below.

Data model, API, roles, permission names and any additional rules are up to you.

Questions Your Design Must Answer

There is no single right answer to any of these. We want to see your reasoning and the trade-offs you chose. If you find a question we didn't ask, ask it and answer it.

1. Should everyone see everything? If not, who sees what? How does that differ from who may act?
2. How would you describe someone who is a Maker for every client but a Checker for only a few? What about someone who is a Checker for one client at one branch, but only for imports?
3. What if the person who prepared a checklist is also qualified to approve it?
4. What if the person responsible for a job is absent?
5. What happens to jobs a person is halfway through when their responsibilities change?
6. Who should be able to cancel a filed job or reopen a closed one? What should it take?
7. What if a client's own employee needs to follow their shipments?
8. What if an admin needs to fix a stuck job? What if an admin tries to make themselves more powerful?
9. How should an automated parser that creates and updates jobs overnight be treated?
10. How do you answer "why can she do this and he can't?"
11. How do you answer "who could have changed this job on the 3rd?"
12. How would a new kind of role be introduced next quarter while engineering is busy?
13. How do you make sure a person's "jobs waiting for me" count matches what they can actually do?
14. What should happen when your system is unsure?

Constraints

- **Scale:** assume 500 users and 200,000 jobs. Lists must stay fast.
- **Timing:** changes to access should take effect when people would expect them to.

Sample Data

We are not providing a dataset. Create your own, small and deliberate: 5 branches, about 10 clients, 20 users and a few hundred jobs across directions, modes and stages. Include the awkward cases on purpose: a Maker everywhere who is a Checker for only a few clients, a Checker limited to one client, one branch and imports only, an absent user, a client-side user, an admin and the overnight parser.

In your README, write each person's access in plain terms: *"Person A has role B for client C, branch D, direction E, mode F."* Also provide a way to generate the scale case (500 users, 200,000 jobs).

Submission

1. **Working code** — a service we can run locally from your README. Any language, framework and storage.
2. **Design document** — 2 to 4 pages: your model, key decisions and trade-offs, and your answer to each question above (plus any you added).
3. **Sample data** — the data you created, loadable with one command, with each person's access written out in the README.
4. **Tests** — automated tests for the key edge cases, such as maker/checker conflict, absent owner, changed responsibility, filed-job cancel or reopen, admin self-escalation and the parser.
5. **Demo** — a recorded or live walkthrough of about 5 minutes: the service running, "why can she do this and he can't?", and a job's history.

> *Where the brief is silent, make a reasonable assumption and write it down. Cut scope if you must, but tell us what you cut and why. Be ready to discuss any part of your submission in a follow-up conversation.*